



Sistema de Monitoreo de Acciones Peligrosas en el Conductor

Iván Augusto Camargo López - 2230033

Santiago Torres - 2202024

Índice

1. Preprocesamiento: Repaso y Compilación del Dataset
2. Aplicación: Algoritmos de Machine Learning
3. Aplicación: Mejorando Algoritmos de Machine Learning
4. Más allá: Buscando Relaciones para mejorar el Modelo.



1. Preprocesamiento: Repaso y Compilación del Dataset



- Conviene realizar una recapitulación rápida de en qué punto se encuentra el progreso del proyecto.
- Por ello, es necesario repasar brevemente como procesamos los datos para que sean interpretados por el algoritmo de Machine Learning seleccionado.

El Dataset: Driver Monitoring Dataset

- El **DMD** completo cuenta con más de **25 TBs** de datos, organizados de acuerdo a las acciones realizadas en cada video.
- Se ha escogido la división **drowsiness** del dataset, utilizando los videos desde la **cámara facial** para identificar signos de fatiga.



vicomtech

MEMBER OF BASQUE RESEARCH
& TECHNOLOGY ALLIANCE

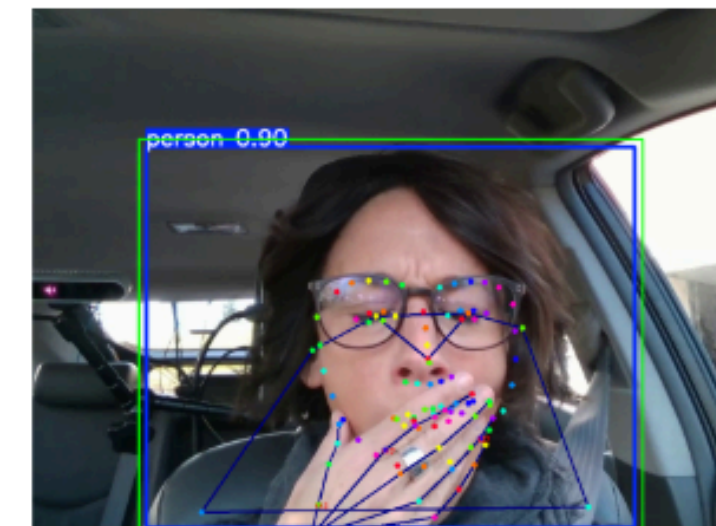
Procesamiento para ML

- A través de **ViTPose** y **Mediapipe** extraemos **landmarks** de puntos clave en el rostro y en las manos.



- Así, no es necesario trabajar con un dataset lleno de imágenes; únicamente con un **dataframe de coordenadas**.

ViTPose



Procesamiento para ML

- Procesando a los **16 sujetos** de la partición del dataset, se termina con un total de **92941 filas** y **1454 columnas**

frame	lmk_1_x	lmk_1_y	lmk_1_z	[...]	lmk_hand_1_x	eyes_state
1	0.453872	0.192839	0.678126	[...]	0.788235	eyes_state/open
2	0.123583	0.12840	0.237893	[...]	NaN	NaN
3	[...]	[...]	[...]	[...]	[...]	eyes_state/close
[...]	[...]	[...]	[...]	[...]	[...]	eyes_state/opening

Procesamiento para ML

- Para poder ingresar procesar el dataframe con métodos de Machine Learning, realizamos un **mapeo de las etiquetas** y reemplazamos los **NaN** por **0** (simbolizando la ausencia de la coordenada)

frame	lmk_1_x	lmk_1_y	lmk_1_z	[...]	lmk_hand_1_x	eyes_state
1	0.453872	0.192839	0.678126	[...]	0.788235	1
2	0.123583	0.12840	0.237893	[...]	0	0
3	[...]	[...]	[...]	[...]	[...]	2
[...]	[...]	[...]	[...]	[...]	[...]	3

Mapeo de Etiquetas

Etiqueta Original	Clave
NaN	0
blinks/blinking	1

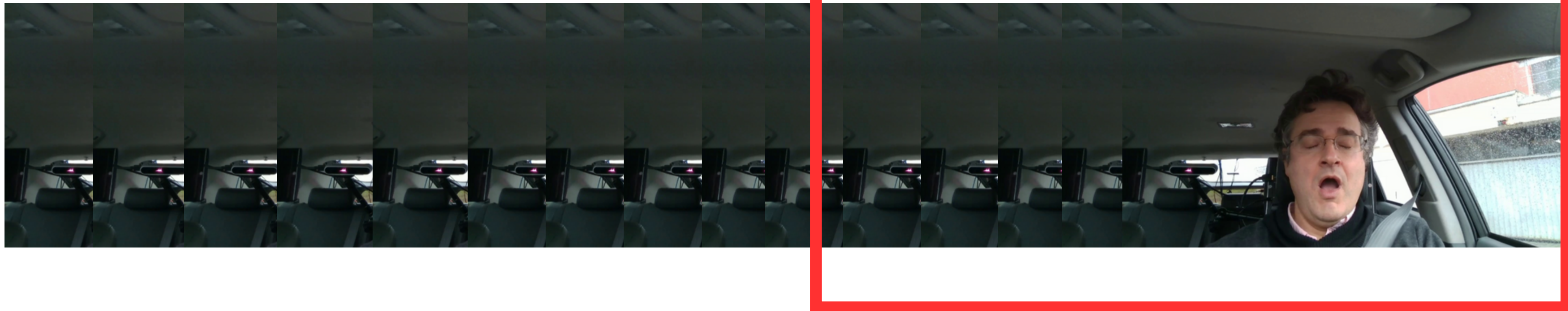
Etiqueta Original	Clave
NaN	0
yawning/Yawning without hand	1
yawning/Yawning with hand	2

Etiqueta Original	Clave
NaN	0
eyes_state/open	1
eyes_state/close	2
eyes_state/opening	3
eyes_state/closing	4
undefined	5

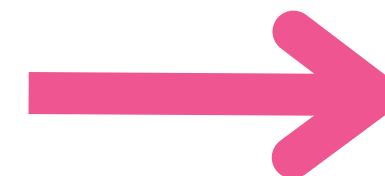
- Se eliminó la columna oclussion, dado que solo contenía NaN*

Uso de las predicciones

- Observar ventanas con predicciones de las etiquetas, y asignar cierta **tolerancia** que dependa de ellas para mandar una **alerta**



$$w0 * \text{blinks_window} + w1 * \text{eyes_state_window} + w2 * \text{yawning_window} > \text{tolerancia_value}$$



2. Aplicación: Algoritmos de Machine Learning

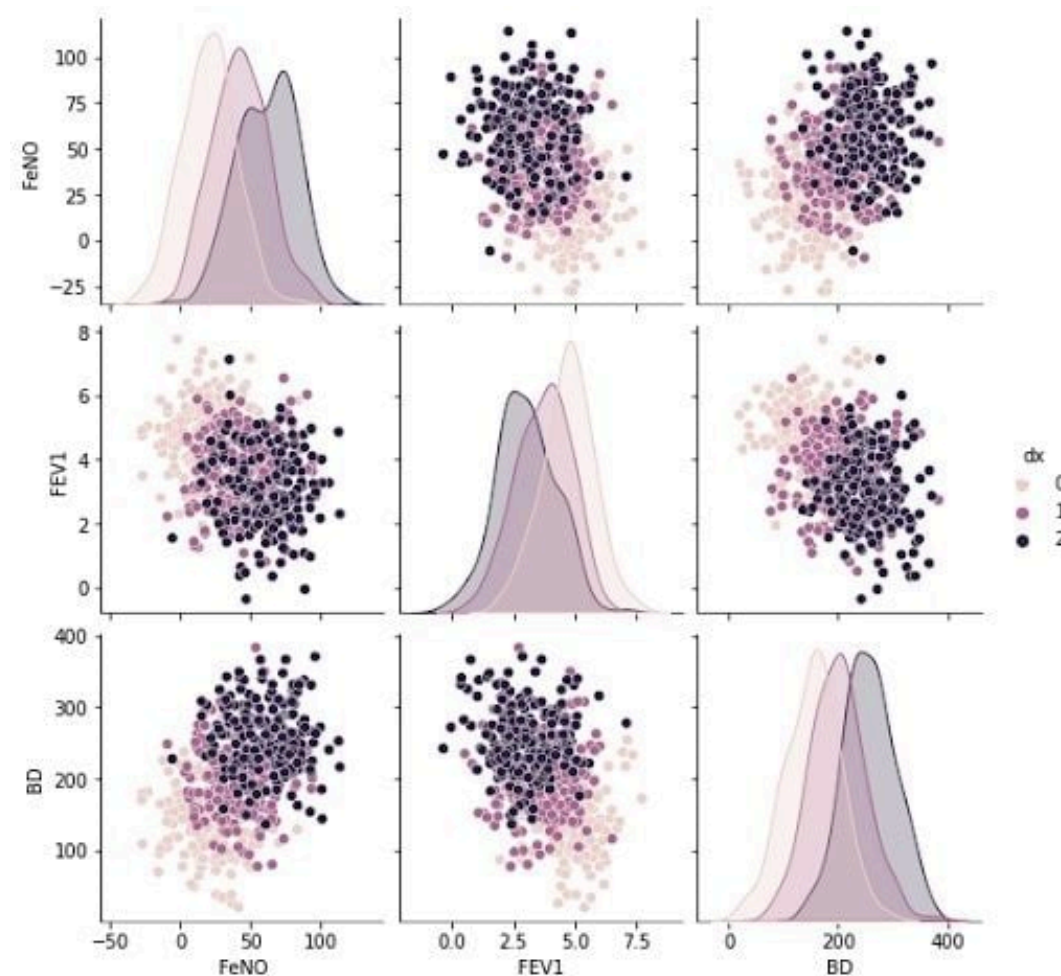
- Con el dataset listo y acondicionado para los algoritmos de machine learning, es el momento de **aplicar cada uno de ellos para evaluar cuál es el mejor**
- Primero, se van a utilizar los **ajustes predeterminados** de cada uno de los métodos de clasificación.



Naive Gaussian Bayes

¿Qué es?

- Un clasificador basado en el Teorema de Bayes, que asume que las características son independientes y siguen una distribución normal (gaussiana) dentro de cada clase.



¿Cómo funciona?

- Calcula la media y desviación estándar de cada característica por clase, luego usa el Teorema de Bayes para predecir la clase con la mayor probabilidad.

Naive Gaussian Bayes - Métricas

- [Tabla del Kfold = 5]

	blinks	eyes_estate	yawning
train_Accuracy	0.639 (+/- 0.00058)	0.233 (+/- 0.06595)	0.130 (+/- 0.00084)
test_Accuracy	0.639 (+/- 0.00481)	0.233 (+/- 0.06669)	0.130 (+/- 0.00438)
train_Precision	0.570 (+/- 0.00092)	0.472 (+/- 0.00311)	0.830 (+/- 0.00063)
test_Precision	0.570 (+/- 0.00753)	0.471 (+/- 0.00964)	0.830 (+/- 0.00504)
train_Recall	0.639 (+/- 0.00058)	0.233 (+/- 0.06595)	0.130 (+/- 0.00084)
test_Recall	0.639 (+/- 0.00481)	0.233 (+/- 0.06669)	0.130 (+/- 0.00438)
train_F1_Score	0.556 (+/- 0.00097)	0.300 (+/- 0.06342)	0.138 (+/- 0.00118)
test_F1_Score	0.556 (+/- 0.00602)	0.299 (+/- 0.06453)	0.138 (+/- 0.00487)
train_Cohen_Kappa	0.024 (+/- 0.00090)	0.005 (+/- 0.00193)	0.012 (+/- 0.00018)
test_Cohen_Kappa	0.024 (+/- 0.00800)	0.004 (+/- 0.00311)	0.012 (+/- 0.00106)

Naive Gaussian Bayes - Métricas

- [Tabla de partición 80-20]

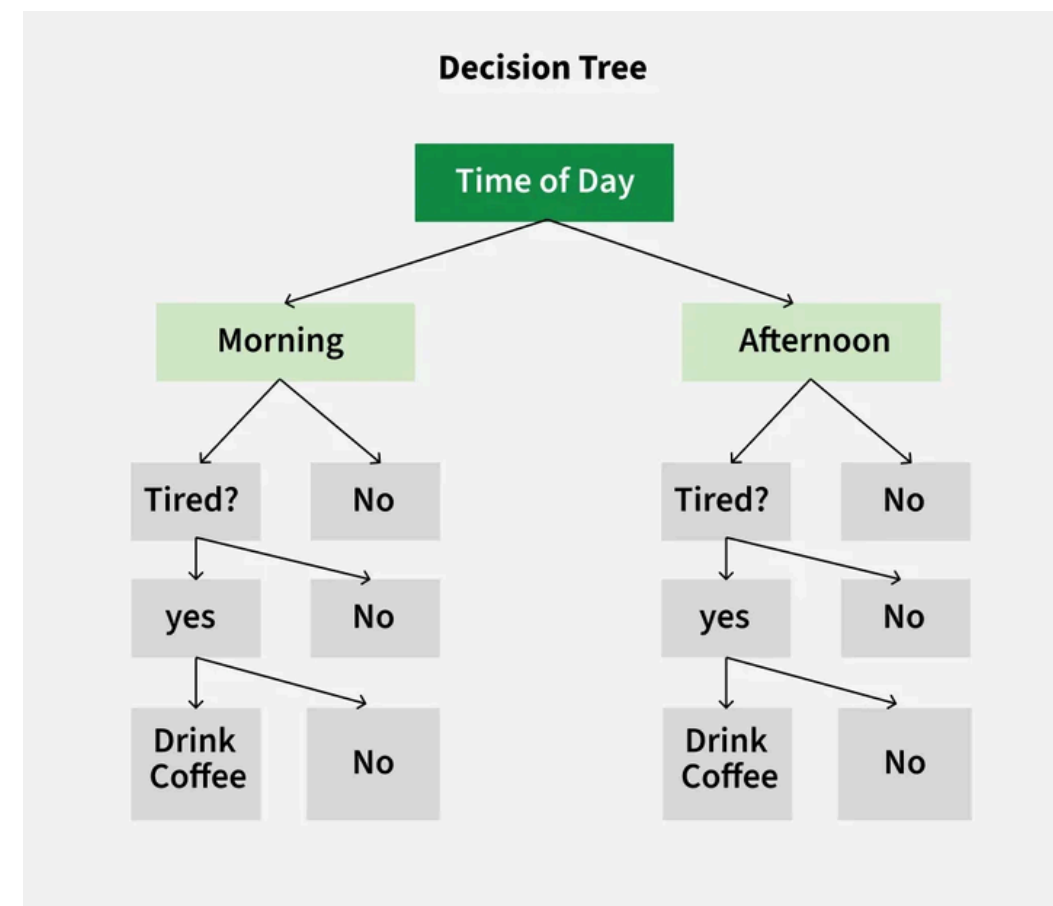
	blinks	eyes_estate	yawning
train_Accuracy	0.638	0.089	0.131
test_Accuracy	0.641	0.088	0.131
train_Precision	0.568	0.481	0.831
test_Precision	0.576	0.482	0.827
train_Recall	0.638	0.089	0.131
test_Recall	0.641	0.088	0.131
train_F1_Score	0.555	0.146	0.140
test_F1_Score	0.560	0.144	0.141
train_Cohen_Kappa	0.022	0.004	0.012
test_Cohen_Kappa	0.030	0.002	0.011

- Funciona bien con datos independientes y distribuciones normales, pero no maneja bien las relaciones no lineales.
- Los datos de los landmarks no obedecen distribuciones normales, o, al menos, no siempre. Además, no considera relaciones entre ellos, ni las importancias de cada uno

Decision Tree Classifier

¿Qué es?

- Un modelo de clasificación que utiliza un árbol de decisiones para dividir los datos en ramas basadas en preguntas sobre las características, hasta llegar a una decisión (hoja).



¿Cómo funciona?

- Divide los datos en subconjuntos utilizando reglas simples, eligiendo características que mejor separen las clases.

Decision Tree Classifier - Métricas

Universidad
Industrial de
Santander



- [Tabla del Kfold = 5]

	blinks	eyes_estate	yawning
train_Accuracy	0.984 (+/- 0.00014)	0.984 (+/- 0.00022)	0.997 (+/- 0.00010)
test_Accuracy	0.774 (+/- 0.00176)	0.704 (+/- 0.00174)	0.934 (+/- 0.00167)
train_Precision	0.984 (+/- 0.00013)	0.984 (+/- 0.00021)	0.997 (+/- 0.00010)
test_Precision	0.773 (+/- 0.00171)	0.704 (+/- 0.00273)	0.933 (+/- 0.00187)
train_Recall	0.984 (+/- 0.00014)	0.984 (+/- 0.00022)	0.997 (+/- 0.00010)
test_Recall	0.774 (+/- 0.00176)	0.704 (+/- 0.00174)	0.934 (+/- 0.00167)
train_F1_Score	0.984 (+/- 0.00014)	0.983 (+/- 0.00022)	0.997 (+/- 0.00010)
test_F1_Score	0.773 (+/- 0.00171)	0.704 (+/- 0.00209)	0.933 (+/- 0.00177)
train_Cohen_Kappa	0.964 (+/- 0.00033)	0.969 (+/- 0.00043)	0.985 (+/- 0.00041)
test_Cohen_Kappa	0.496 (+/- 0.00696)	0.442 (+/- 0.00424)	0.703 (+/- 0.00992)

Decision Tree Classifier - Métricas



- [Tabla de partición 80-20]

	blinks	eyes_estate	yawning
train_Accuracy	0.984	0.984	0.997
test_Accuracy	0.772	0.709	0.940
train_Precision	0.984	0.984	0.997
test_Precision	0.771	0.709	0.940
train_Recall	0.984	0.984	0.997
test_Recall	0.772	0.709	0.940
train_F1_Score	0.984	0.984	0.997
test_F1_Score	0.772	0.709	0.940
train_Cohen_Kappa	0.964	0.969	0.985
test_Cohen_Kappa	0.493	0.451	0.730

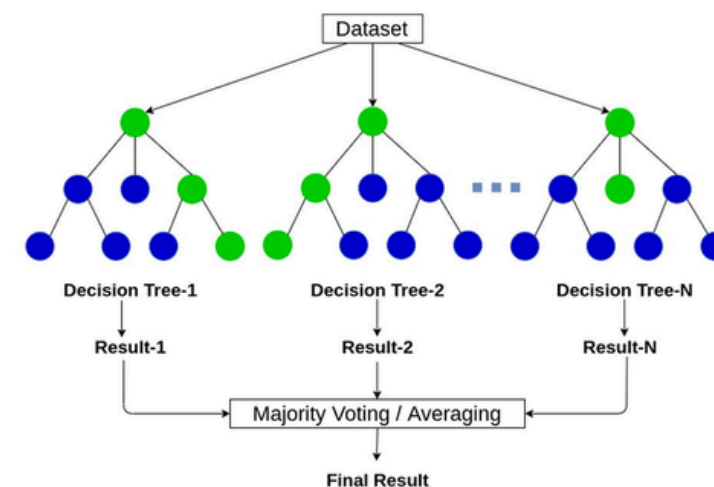
- Fácil de interpretar, pero propenso al sobreajuste si no se ajustan los parámetros correctamente.
- Dado que contamos con muchos parámetros numéricos, un Decision Tree puede encontrar varios thresholds a través de los valores de las features que considere correctos.

Random Forest Classifier

¿Qué es?

- Un modelo de ensamble que utiliza varios árboles de decisión para hacer predicciones. Cada árbol toma decisiones basadas en un subconjunto aleatorio de los datos y características.

Random Forest



¿Cómo funciona?

- Cada árbol se entrena de manera independiente y se combinan los resultados para obtener una predicción final (votación para clasificación).

Random Forest Classifier- Métricas

- [Tabla del Kfold = 5]

	blinks	eyes_estate	yawning
train_Accuracy	0.984 (+/- 0.00024)	0.984 (+/- 0.00016)	0.997 (+/- 0.00011)
test_Accuracy	0.859 (+/- 0.00244)	0.807 (+/- 0.00241)	0.966 (+/- 0.00156)
train_Precision	0.984 (+/- 0.00023)	0.984 (+/- 0.00016)	0.997 (+/- 0.00011)
test_Precision	0.859 (+/- 0.00239)	0.799 (+/- 0.00298)	0.966 (+/- 0.00155)
train_Recall	0.984 (+/- 0.00024)	0.984 (+/- 0.00016)	0.997 (+/- 0.00011)
test_Recall	0.859 (+/- 0.00244)	0.807 (+/- 0.00241)	0.966 (+/- 0.00156)
train_F1_Score	0.984 (+/- 0.00024)	0.983 (+/- 0.00017)	0.997 (+/- 0.00011)
test_F1_Score	0.855 (+/- 0.00260)	0.787 (+/- 0.00231)	0.964 (+/- 0.00175)
train_Cohen_Kappa	0.964 (+/- 0.00055)	0.969 (+/- 0.00031)	0.985 (+/- 0.00051)
test_Cohen_Kappa	0.673 (+/- 0.00522)	0.586 (+/- 0.00540)	0.830 (+/- 0.00938)

Random Forest Classifier-Métricas

- [Tabla de partición 80-20]

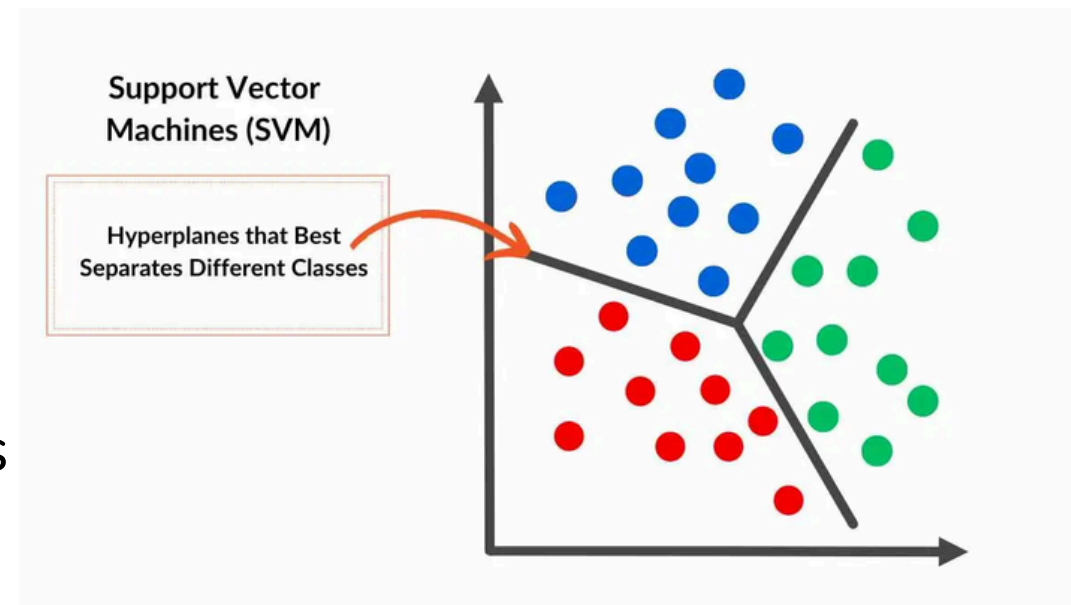
	blinks	eyes_estate	yawning
train_Accuracy	0.984	0.984	0.997
test_Accuracy	0.861	0.806	0.969
train_Precision	0.984	0.984	0.997
test_Precision	0.862	0.799	0.969
train_Recall	0.984	0.984	0.997
test_Recall	0.861	0.806	0.969
train_F1_Score	0.984	0.984	0.997
test_F1_Score	0.857	0.785	0.967
train_Cohen_Kappa	0.964	0.969	0.985
test_Cohen_Kappa	0.679	0.583	0.843

- Modelo robusto que combina múltiples árboles, reduce sobreajuste y maneja relaciones no lineales eficazmente.
- La presencia de múltiples árboles le permite conseguir valores muy altos de precisión. Rinde especialmente bien para las clasificaciones no binarias.

Support Vector Machine (SVM)

¿Qué es?

- Un modelo de clasificación que encuentra el hiperplano que mejor separa las clases en el espacio de características, maximizando el margen entre las clases.



¿Cómo funciona?

- Utiliza vectores de soporte (puntos cercanos al margen) para determinar el hiperplano óptimo que separa las clases.

Support Vector Machine- Métricas

- [Tabla del Kfold = 5]

	blinks	eyes_estate	yawning
train_Accuracy	0.496 (+/- 0.08355)	0.360 (+/- 0.05565)	0.480 (+/- 0.19594)
test_Accuracy	0.499 (+/- 0.08217)	0.360 (+/- 0.05678)	0.479 (+/- 0.19554)
train_Precision	0.541 (+/- 0.02227)	0.420 (+/- 0.00855)	0.776 (+/- 0.01844)
test_Precision	0.545 (+/- 0.02465)	0.420 (+/- 0.01087)	0.775 (+/- 0.01806)
train_Recall	0.496 (+/- 0.08355)	0.360 (+/- 0.05565)	0.480 (+/- 0.19594)
test_Recall	0.499 (+/- 0.08217)	0.360 (+/- 0.05678)	0.479 (+/- 0.19554)
train_F1_Score	0.452 (+/- 0.10491)	0.382 (+/- 0.03563)	0.537 (+/- 0.22686)
test_F1_Score	0.455 (+/- 0.10459)	0.381 (+/- 0.03709)	0.536 (+/- 0.22667)
train_Cohen_Kappa	-0.022 (+/- 0.03145)	-0.024 (+/- 0.00899)	-0.012 (+/- 0.00754)
test_Cohen_Kappa	-0.017 (+/- 0.03309)	-0.024 (+/- 0.00948)	-0.013 (+/- 0.00729)

Support Vector Machine-Métricas

- [Tabla de partición 80-20]

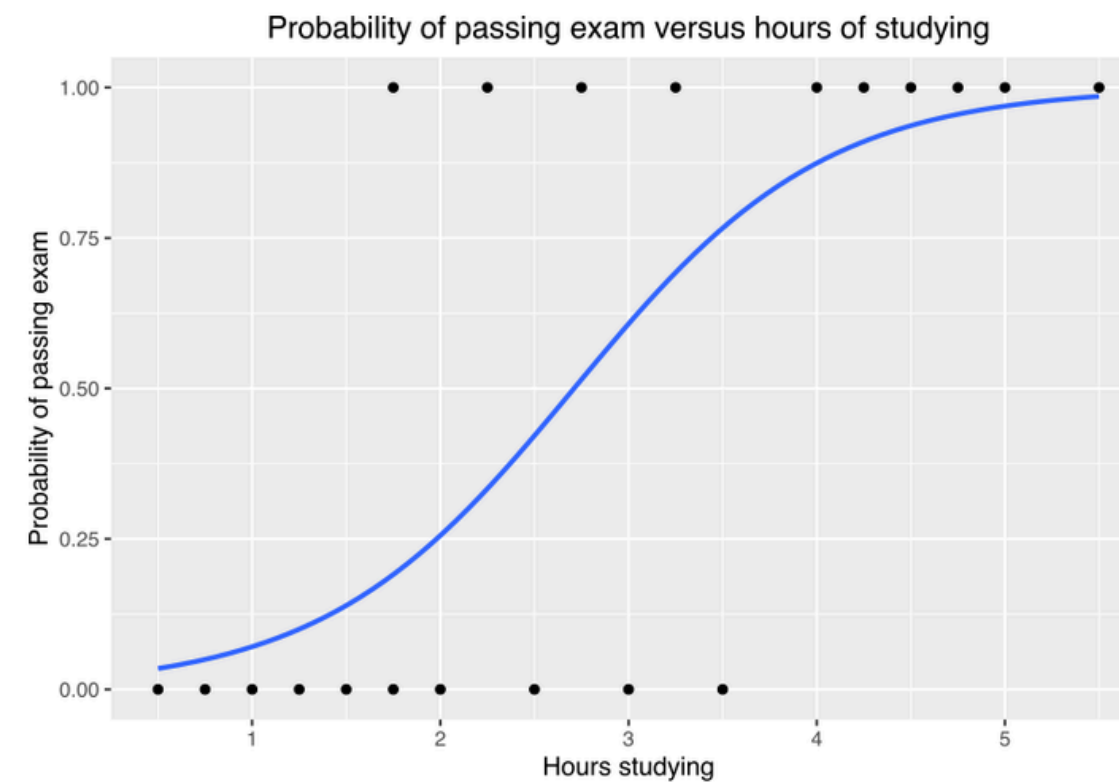
	blinks	eyes_estate	yawning
train_Accuracy	0.657	0.652	0.875
test_Accuracy	0.658	0.653	0.878
train_Precision	0.645	0.426	0.766
test_Precision	0.662	0.426	0.771
train_Recall	0.657	0.652	0.875
test_Recall	0.658	0.653	0.878
train_F1_Score	0.531	0.515	0.817
test_F1_Score	0.532	0.515	0.821
train_Cohen_Kappa	0.017	0.000	0.000
test_Cohen_Kappa	0.020	0.000	0.000

- Eficaz en espacios de alta dimensión, pero requiere mucho tiempo de cómputo con grandes conjuntos de datos.
- Ante tantos vectores [algunos muy similares entre sí], se dificulta encontrar hiperplanos para realizar las divisiones de forma correcta.
- Además, es demasiado costoso computacionalmente.

Logistic Regression

¿Qué es?

- Un modelo de clasificación que estima la probabilidad de que una instancia pertenezca a una clase utilizando una función logística.



¿Cómo funciona?

- Calcula una función sigmoide para predecir la probabilidad de las clases y luego asigna la clase con la probabilidad más alta

Logistic Regression- Métricas

- [Tabla del Kfold = 5]

	blinks	eyes_estate	yawning
train_Accuracy	0.658 (+/- 0.00078)	0.652 (+/- 0.00082)	0.876 (+/- 0.00045)
test_Accuracy	0.657 (+/- 0.00315)	0.652 (+/- 0.00259)	0.876 (+/- 0.00173)
train_Precision	0.643 (+/- 0.00393)	0.504 (+/- 0.02259)	0.817 (+/- 0.00645)
test_Precision	0.631 (+/- 0.00910)	0.479 (+/- 0.02091)	0.815 (+/- 0.00856)
train_Recall	0.658 (+/- 0.00078)	0.652 (+/- 0.00082)	0.876 (+/- 0.00045)
test_Recall	0.657 (+/- 0.00315)	0.652 (+/- 0.00259)	0.876 (+/- 0.00173)
train_F1_Score	0.534 (+/- 0.00100)	0.516 (+/- 0.00115)	0.818 (+/- 0.00067)
test_F1_Score	0.532 (+/- 0.00412)	0.516 (+/- 0.00316)	0.818 (+/- 0.00256)
train_Cohen_Kappa	0.020 (+/- 0.00110)	0.002 (+/- 0.00089)	0.002 (+/- 0.00028)
test_Cohen_Kappa	0.018 (+/- 0.00256)	0.002 (+/- 0.00054)	0.002 (+/- 0.00059)

Logistic Regression-Métricas

- [Tabla de partición 80-20]

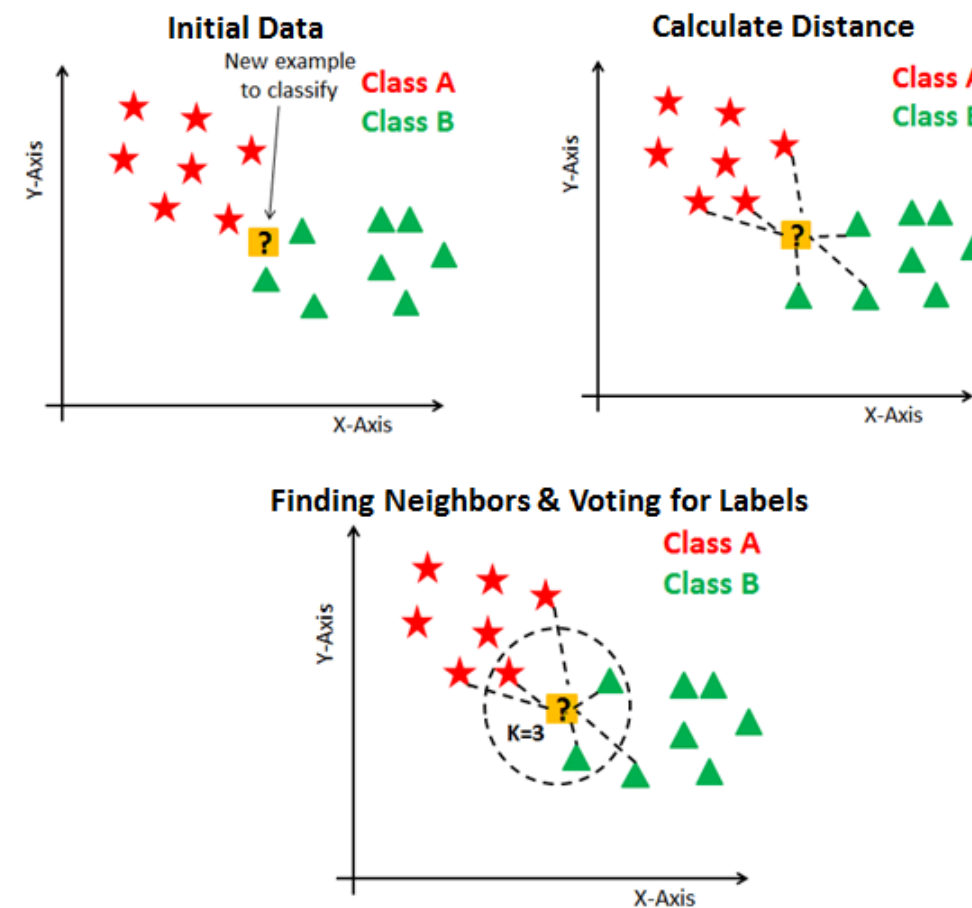
	blinks	eyes_estate	yawning
train_Accuracy	0.657	0.652	0.875
test_Accuracy	0.658	0.652	0.878
train_Precision	0.637	0.475	0.811
test_Precision	0.646	0.481	0.815
train_Recall	0.657	0.652	0.875
test_Recall	0.658	0.652	0.878
train_F1_Score	0.534	0.516	0.818
test_F1_Score	0.535	0.516	0.822
train_Cohen_Kappa	0.020	0.001	0.002
test_Cohen_Kappa	0.023	0.001	0.002

- Rápido y eficiente para problemas lineales, pero no funciona bien con relaciones complejas o no lineales.
- Es nuestro caso, pues los landmarks no cambian de manera lineal. Existen rotaciones, inclinaciones, gestos, etc.

Kneighbors Classifier

¿Qué es?

- Un modelo de clasificación que asigna la clase más común entre los k vecinos más cercanos a un punto de datos.



¿Cómo funciona?

- Calcula la distancia entre el punto de datos y sus vecinos más cercanos, y asigna la clase que es más frecuente entre esos vecinos.

Kneighbors Classifier- Métricas

- [Tabla del Kfold = 5]

	blinks	eyes_estate	yawning
train_Accuracy	0.901 (+/- 0.00063)	0.861 (+/- 0.00074)	0.980 (+/- 0.00024)
test_Accuracy	0.846 (+/- 0.00331)	0.785 (+/- 0.00178)	0.968 (+/- 0.00129)
train_Precision	0.901 (+/- 0.00062)	0.857 (+/- 0.00073)	0.979 (+/- 0.00025)
test_Precision	0.844 (+/- 0.00341)	0.767 (+/- 0.00229)	0.968 (+/- 0.00132)
train_Recall	0.901 (+/- 0.00063)	0.861 (+/- 0.00074)	0.980 (+/- 0.00024)
test_Recall	0.846 (+/- 0.00331)	0.785 (+/- 0.00178)	0.968 (+/- 0.00129)
train_F1_Score	0.900 (+/- 0.00067)	0.854 (+/- 0.00076)	0.979 (+/- 0.00025)
test_F1_Score	0.844 (+/- 0.00336)	0.769 (+/- 0.00225)	0.968 (+/- 0.00127)
train_Cohen_Kappa	0.778 (+/- 0.00162)	0.722 (+/- 0.00139)	0.907 (+/- 0.00117)
test_Cohen_Kappa	0.652 (+/- 0.00799)	0.562 (+/- 0.00512)	0.854 (+/- 0.00553)

Kneighbors Classifier-Métricas

- [Tabla de partición 80-20]

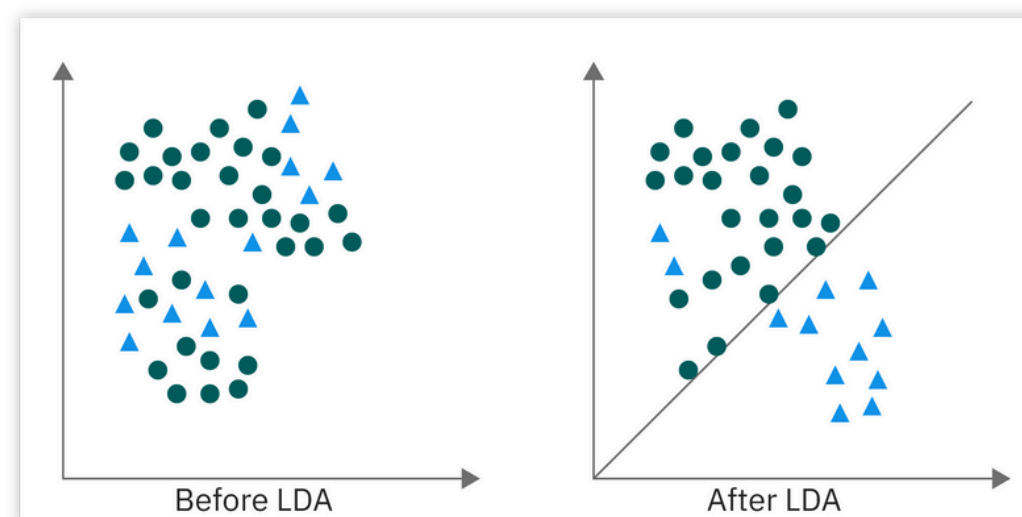
	blinks	eyes_estate	yawning
train_Accuracy	0.903	0.863	0.979
test_Accuracy	0.843	0.789	0.969
train_Precision	0.903	0.858	0.979
test_Precision	0.841	0.772	0.968
train_Recall	0.903	0.863	0.979
test_Recall	0.843	0.789	0.969
train_F1_Score	0.902	0.855	0.979
test_F1_Score	0.842	0.773	0.968
train_Cohen_Kappa	0.781	0.724	0.906
test_Cohen_Kappa	0.647	0.570	0.854

- Simple, pero costoso computacionalmente con grandes datos; funciona bien con datos no lineales y distribuciones locales.
- Es especialmente útil, dado que relaciona los datos con sus vecinos próximos. En otras palabras, ayuda a relacionar las coordenadas de los puntos cercanos.

Linear Discriminant Analysis

¿Qué es?

- Un modelo de clasificación que busca encontrar una proyección lineal de las características para maximizar la separación entre las clases.



¿Cómo funciona?

- Calcula la media y varianza de las características dentro de cada clase, y luego proyecta los datos en una nueva dimensión para mejorar la clasificación.

Linear Discriminant Analysis-Métricas

- [Tabla del Kfold = 5]

	blinks	eyes_estate	yawning
train_Accuracy	0.663 (+/- 0.00055)	0.651 (+/- 0.00105)	0.880 (+/- 0.00039)
test_Accuracy	0.659 (+/- 0.00101)	0.649 (+/- 0.00298)	0.879 (+/- 0.00212)
train_Precision	0.634 (+/- 0.00172)	0.593 (+/- 0.00138)	0.850 (+/- 0.00068)
test_Precision	0.622 (+/- 0.00278)	0.568 (+/- 0.00540)	0.847 (+/- 0.00331)
train_Recall	0.663 (+/- 0.00055)	0.651 (+/- 0.00105)	0.880 (+/- 0.00039)
test_Recall	0.659 (+/- 0.00101)	0.649 (+/- 0.00298)	0.879 (+/- 0.00212)
train_F1_Score	0.579 (+/- 0.00081)	0.536 (+/- 0.00130)	0.853 (+/- 0.00055)
test_F1_Score	0.574 (+/- 0.00108)	0.533 (+/- 0.00446)	0.850 (+/- 0.00247)
train_Cohen_Kappa	0.080 (+/- 0.00186)	0.052 (+/- 0.00074)	0.241 (+/- 0.00334)
test_Cohen_Kappa	0.069 (+/- 0.00292)	0.045 (+/- 0.00318)	0.229 (+/- 0.00961)

Linear Discriminant Analysis-Métricas

- [Tabla de partición 80-20]

	blinks	eyes_estate	yawning
train_Accuracy	0.662	0.650	0.880
test_Accuracy	0.660	0.650	0.882
train_Precision	0.631	0.589	0.850
test_Precision	0.625	0.588	0.850
train_Recall	0.662	0.650	0.880
test_Recall	0.660	0.650	0.882
train_F1_Score	0.578	0.536	0.852
test_F1_Score	0.574	0.535	0.853
train_Cohen_Kappa	0.077	0.052	0.243
test_Cohen_Kappa	0.071	0.052	0.227

- Eficiente para clasificación lineal, pero limitado cuando las clases no siguen distribuciones normales.
- Nuevamente, dado que es un método lineal aplicado a datos con relaciones no lineales, no es eficiente.

Deep Learning - Métricas

- [Tabla del Kfold = 5]

	blinks	eyes_estate	yawning
train_Accuracy	0.657 (+/- 0.00070)	0.654 (+/- 0.00076)	0.876 (+/- 0.00057)
test_Accuracy	0.657 (+/- 0.00275)	0.653 (+/- 0.00319)	0.876 (+/- 0.00225)
train_Precision	0.672 (+/- 0.02208)	0.640 (+/- 0.00744)	0.826 (+/- 0.00171)
test_Precision	0.647 (+/- 0.00869)	0.574 (+/- 0.01420)	0.824 (+/- 0.00431)
train_Recall	0.657 (+/- 0.00070)	0.654 (+/- 0.00076)	0.876 (+/- 0.00057)
test_Recall	0.657 (+/- 0.00275)	0.653 (+/- 0.00319)	0.876 (+/- 0.00225)
train_F1_Score	0.528 (+/- 0.00206)	0.520 (+/- 0.00135)	0.819 (+/- 0.00082)
test_F1_Score	0.527 (+/- 0.00336)	0.518 (+/- 0.00384)	0.819 (+/- 0.00333)
train_Cohen_Kappa	0.014 (+/- 0.00168)	0.010 (+/- 0.00176)	0.007 (+/- 0.00058)
test_Cohen_Kappa	0.012 (+/- 0.00301)	0.006 (+/- 0.00167)	0.006 (+/- 0.00140)

Deep Learning -Métricas

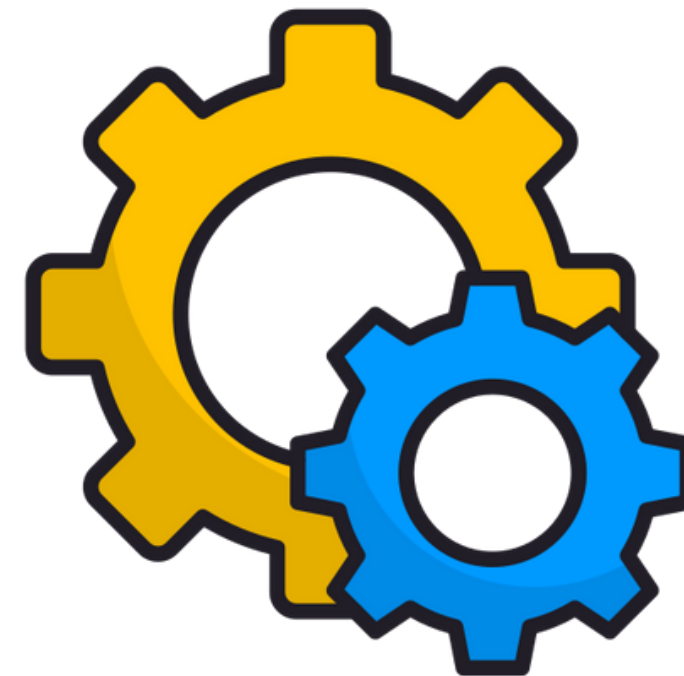
- [Tabla de partición 80-20]

	blinks	eyes_estate	yawning
train_Accuracy	0.657	0.654	0.876
test_Accuracy	0.656	0.653	0.879
train_Precision	0.702	0.654	0.822
test_Precision	0.669	0.573	0.823
train_Recall	0.657	0.654	0.876
test_Recall	0.656	0.653	0.879
train_F1_Score	0.526	0.520	0.818
test_F1_Score	0.524	0.518	0.823
train_Cohen_Kappa	0.012	0.010	0.007
test_Cohen_Kappa	0.008	0.006	0.010

- Aprende patrones no lineales, pero esta representación inicial es una red neuronal fully connected muy vaga.
- Solo se usó una configuración básica con 1 capa oculta en este ejemplo

3. Aplicación: Mejorando Algoritmos de Machine Learning

- Con el fin de adaptar los modelos de una mejor forma al dataset, se hace necesario modificar los **hiperparámetros** inherentes a cada uno.
- Con la modificación de estos parámetros, se espera mejorar las métricas calculadas.



Resultados de Modificación de Parámetros

DecisionTreeClassifier

```
max_depth=100,  
min_samples_split=100,  
min_samples_leaf=50,  
max_features='log2',  
criterion='gini',  
class_weight='balanced'
```

RandomForestClassifier

```
n_estimators=200,  
max_depth=30,  
min_samples_split=4,  
min_samples_leaf=2,  
max_features='sqrt',  
criterion='gini',  
class_weight='balanced',  
random_state=42,  
n_jobs=-1,  
oob_score=True
```

SupportVectorMachine

```
kernel='linear',  
C=2.0,  
gamma='scale',  
class_weight='balanced',  
random_state=42,  
max_iter=200,  
tol=1e-3
```

Resultados de Modificación de Parámetros

LogisticRegression

```
max_iter=1000,  
solver='liblinear',  
class_weight='balanced',  
random_state=42,  
C=1.0,  
tol=1e-4
```

KNeighborsClassifier

```
n_neighbors=10,  
weights='uniform',  
metric='minkowski',  
n_jobs=-1
```

RandomForestClassifier

```
n_estimators=150,  
max_depth=25,  
min_samples_split=4,  
min_samples_leaf=2,  
max_features='sqrt',  
criterion='gini',  
class_weight='balanced',  
random_state=42,  
n_jobs=-1,  
oob_score=True
```

Resultados de Modificación de Parámetros

DecisionTreeClassifier

```
max_depth=100,  
min_samples_split=100,  
min_samples_leaf=50,  
max_features='log2',  
criterion='gini',  
class_weight='balanced'
```

KNeighborsClassifier

```
n_neighbors=10,  
weights='uniform',  
metric='minkowski',  
n_jobs=-1
```

Deep Learning

```
Dense(64, activation='relu')  
Dense(128, activation='relu')  
Dense(256, activation='relu')  
Dense(512, activation='relu')  
optimizer='adam',  
loss='sparse_categorical_crossentropy'
```

Y muchos otros más...



11h 58m 28s

Top Mejores Resultados



RandomForestClassifier

```
max_depth=200,  
min_samples_split=2,  
min_samples_leaf=1,  
criterion='gini',  
class_weight='balanced'
```

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Accuracy	<u>0.863</u>	<u>0.810</u>	<u>0.970</u>
train_Precision	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Precision	<u>0.863</u>	<u>0.803</u>	<u>0.970</u>
train_Recall	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Recall	<u>0.863</u>	<u>0.810</u>	<u>0.970</u>
train_F1_Score	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_F1_Score	<u>0.859</u>	<u>0.789</u>	<u>0.968</u>
train_Cohen_Kappa	<u>0.964</u>	<u>0.969</u>	<u>0.985</u>
test_Cohen_Kappa	<u>0.682</u>	<u>0.591</u>	<u>0.847</u>

Top Mejores Resultados



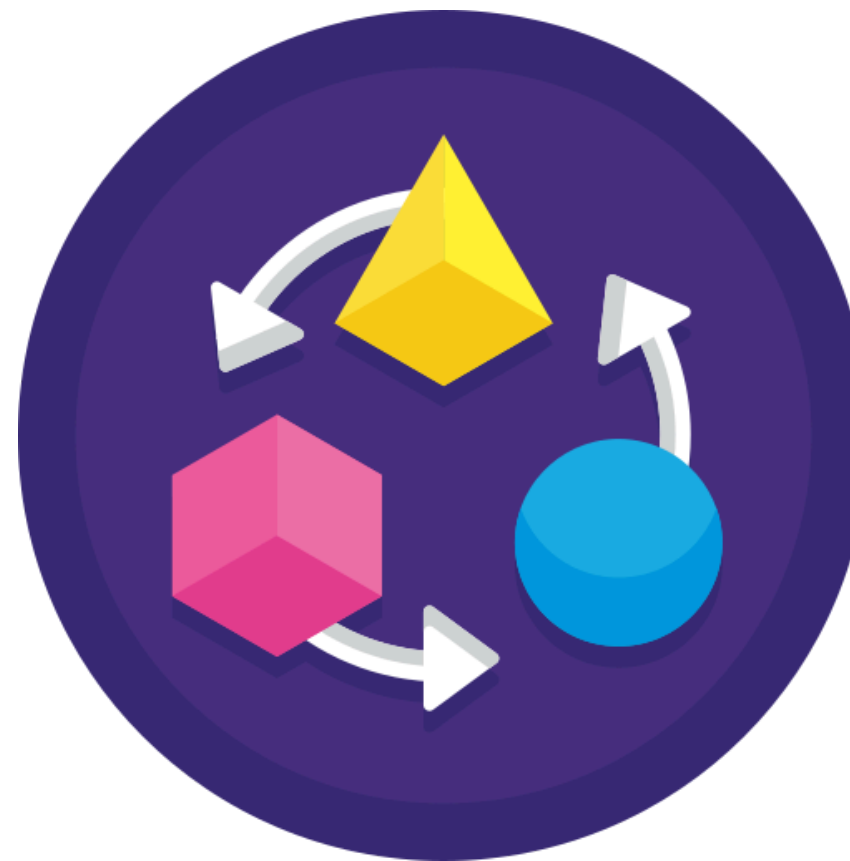
KNeighborsClassifier

n_neighbors=10,
weights='distance',
leaf_size = 50,
n_jobs=-1

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Accuracy	<u>0.847</u>	<u>0.795</u>	<u>0.968</u>
train_Precision	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Precision	<u>0.845</u>	<u>0.779</u>	<u>0.967</u>
train_Recall	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Recall	<u>0.847</u>	<u>0.795</u>	<u>0.968</u>
train_F1_Score	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_F1_Score	<u>0.844</u>	<u>0.780</u>	<u>0.967</u>
train_Cohen_Kappa	<u>0.964</u>	<u>0.969</u>	<u>0.985</u>
test_Cohen_Kappa	<u>0.652</u>	<u>0.580</u>	<u>0.846</u>

4. Más allá: Buscando relaciones para mejorar el Modelo.

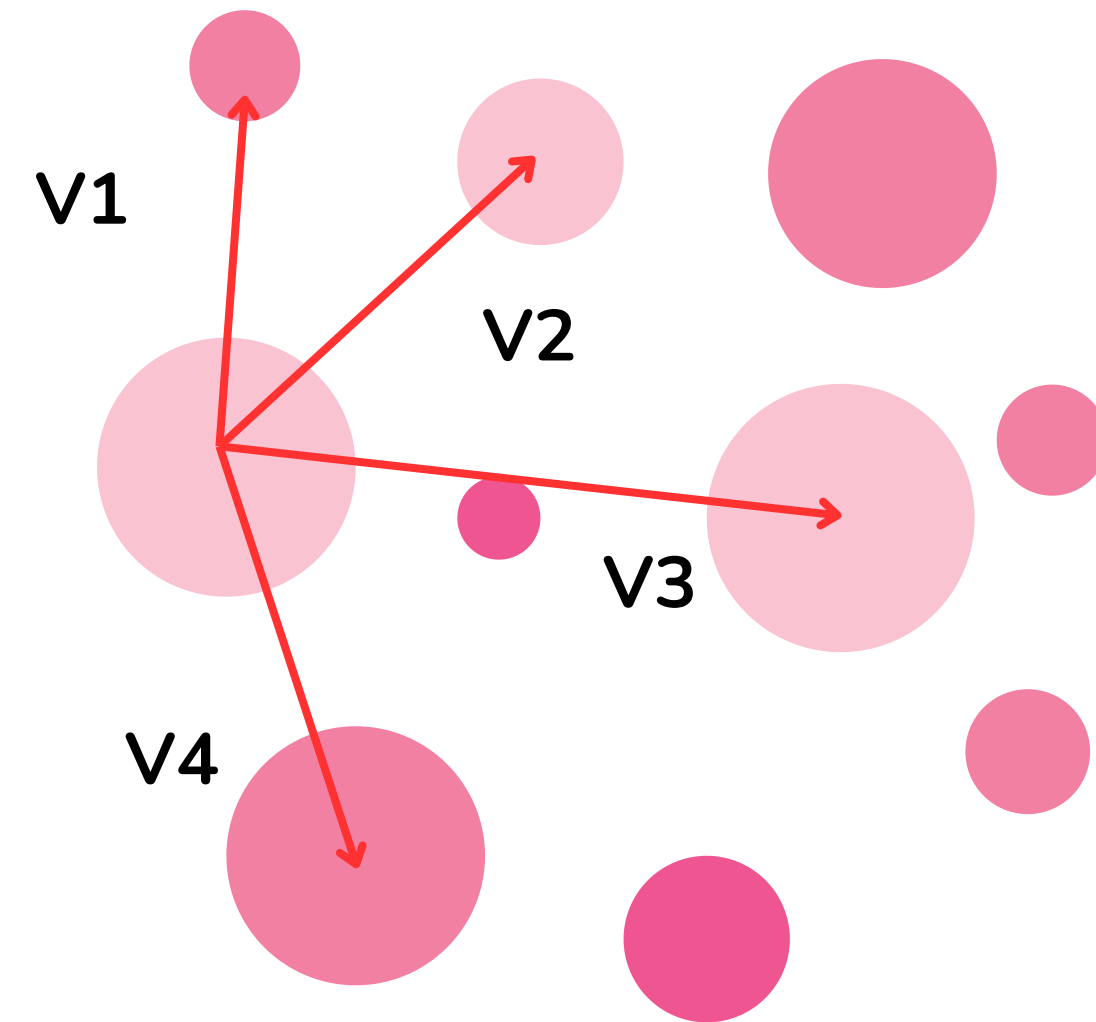
- Si bien se han logrado resultados aceptables, todavía existe un margen de mejora, especialmente para predecir el **eyes_state**



- Para ello, se puede optar por **transformar las coordenadas del dataset** nuevamente antes de ingresarlas a los métodos de Machine Learning

4.1 - Vectores - Diferencia

- Si tratamos a las coordenadas como vectores, hay varias transformaciones por aplicar a los datos iniciales.
- La más sencilla es trazar vectores de diferencia entre coordenadas para obtener un nuevo dataframe
- Para ello, se tomaron los **20 landmarks** más importantes de acuerdo a las importancias descritas por los árboles, se seleccionaron pares, y luego, las diferencias entre ellos.
- Luego, en nuestro método ganador de ML, se ingresó el nuevo dataset con **92491 filas y 570 columnas**



4.1 - Vectores - Diferencia

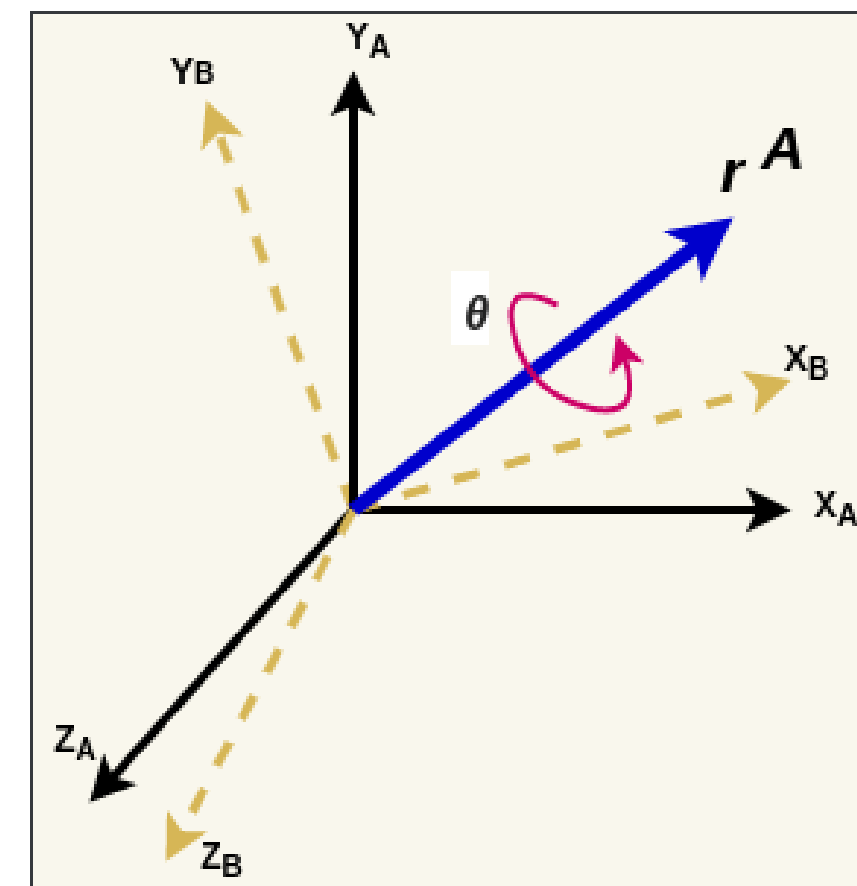
- En general, no se consiguió mejorar ninguna métrica.
- Lo más probable es que se deba a la menor cantidad de features disponibles.
- Utilizando un RandomForest:

	blinks	eyes_estate	yawning
train_Accuracy	0.975	0.975	0.996
test_Accuracy	0.809	0.757	0.944
train_Precision	0.976	0.976	0.996
test_Precision	0.810	0.753	0.945
train_Recall	0.975	0.975	0.996
test_Recall	0.809	0.757	0.944
train_F1_Score	0.975	0.975	0.996
test_F1_Score	0.799	0.718	0.938
train_Cohen_Kappa	0.944	0.952	0.980
test_Cohen_Kappa	0.545	0.437	0.693

4.2 - Cuaterniones

- Un cuaternión es una extensión de los números complejos, representado como:
- Los cuaterniones representan rotaciones. Evitan problemas como el **gimbal lock**, y son herramientas matemáticas eficientes y estables
- En este caso, podemos calcular los **cuaterniones de los vectores diferencia**, con respecto a una referencia, que, en este caso, se consideró como el **eje z** ([0,0,1]).

$$q = w + xi + yj + zk$$



4.2 - Cuaterniones

- En este caso, utilizando los **20 landmarks más importantes**, se obtiene un dataset con **92491 filas y 760 columnas**
- En este caso, utilizando un RandomForest, impresiona la forma en que, **con menos features, obtenemos resultados muy similares.**
- No obstante, no se superaron los récords anteriores propuestos.

	blinks	eyes_estate	yawning
train_Accuracy	0.975	0.975	0.996
test_Accuracy	0.809	0.758	0.945
train_Precision	0.976	0.976	0.996
test_Precision	0.811	0.754	0.945
train_Recall	0.975	0.975	0.996
test_Recall	0.809	0.758	0.945
train_F1_Score	0.975	0.975	0.996
test_F1_Score	0.799	0.719	0.939
train_Cohen_Kappa	0.944	0.952	0.980
test_Cohen_Kappa	0.545	0.440	0.696

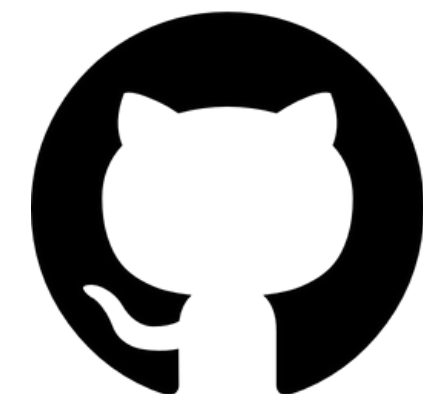
Tablero y Repositorio

Tablero / Implementación



- <https://colab.research.google.com/drive/1k8VJtLAKs58UR6CDE8fYobNOjEDjQgGS?usp=sharing>

Repositorio



- https://github.com/popcorner893/Sistema_Monitoreo_Conductor



¡Gracias!